INFORME TÉCNICO DE SOFTWARE (SDLC)

ATLAS GYM · AUDITORÍA Y PLAN DE PRODUCCIÓN

Diagnóstico Integral de Madurez, Vulnerabilidades, Seguridad de Datos y Roadmap

PROYECTO	ARQUITECTURA	ESTADO GENERAL	TIEMPO A PRODUCCIÓN
ATLAS Gym Platform	**Fullstack (Next.js + Spring Boot + MySQL)**	**Fase de Endurecimiento (Hardening)**	**2 a 3 Semanas (Estimado)**

Dictamen Ejecutivo: La aplicación cuenta con un grado de madurez funcional sobresaliente (**85% completado**), resolviendo de manera exitosa los flujos de socios, finanzas, rutinas, videoteca biomecánica, control de accesos por cuota y autenticación unificada. Para garantizar un lanzamiento en producción **100% seguro**, se requiere completar la fase de blindaje operativo (hardening): rate limiting contra ataques de fuerza bruta, migración de base de datos con versionado formal (Flyway), cookies seguras HttpOnly, variables de entorno aisladas y despliegue bajo HTTPS con respaldos automáticos.

1. ¿En dónde estamos parados en el Ciclo de Vida (SDLC)?

En el modelo de ciclo de vida de desarrollo de software (SDLC), el proyecto se encuentra en la **transición entre la Fase de Integración Fullstack y la Fase de Pre-Lanzamiento / Endurecimiento (Staging & Hardening)**.

Completitud Funcional (Features de Negocio) — 85%

Módulos CRUD, cobro de cuotas, rutinas interactivas, tótem y frontend validados con 130 tests automatizados.

Madurez para Producción (Seguridad y DevOps) — 65%

Entorno actual optimizado para desarrollo local; pendiente configuración de infraestructura segura y despliegue productivo.

2. Estado Detallado por Módulos y Capacidades

Módulo / Componente	Tecnología	Estado	Diagnóstico y Observaciones
Gestión de Socios	Next.js / Spring / JPA	Listo (95%)	Altas, bajas lógicas, búsqueda en vivo, normalización de DNI y teléfono, cálculo de estado de cuota en sala.
Finanzas y Pagos	Next.js / MySQL / WhatsApp	Listo (90%)	Registro de ingresos, historial de pagos, filtros por estado (Al día / Pendiente / Moroso), integración WhatsApp con pestaña fija.
Rutinas y Videoteca	Next.js / Tailwind CSS	Listo (90%)	Asignación individual y múltiple a socios, loops GIF biomecánicos instantáneos, catálogo por grupos musculares.

Módulo / Componente	Tecnología	Estado	Diagnóstico y Observaciones
Tótem de Asistencias	Next.js / Spring Boot	**Listo (90%)**	Validación de ingreso por DNI, bloqueo inteligente a socios con cuota vencida, aforo en sala en tiempo real.
Login y Control de Acceso	Next.js / JWT / Spring Sec	**Listo (85%)**	Módulo unificado para Admin y Alumno con autodetección de permisos, modal seguro para olvido de clave y credenciales robustas.
Blindaje de Seguridad	Spring Security / OWASP	**En proceso (65%)**	Requiere rate limiting anti-fuerza bruta, auditoría de hash BCrypt y protección estricta contra ataques de red.
Infraestructura y DevOps	Docker / SSL / Linux VPS	**Pendiente (40%)**	Faltan Dockerfiles, proxy inverso Nginx con SSL/TLS Let's Encrypt y scripts de respaldo automatizado diario.

3. Auditoría de Seguridad y Protección de Datos (OWASP)

Para que una plataforma que gestiona datos personales y pagos sea jurídicamente sólida y técnicamente inviolable, debe cumplir con las directrices internacionales del **OWASP Top 10** y la legislación local de protección de datos (Ley Nacional N° 25.326).

A. Prevención de Ataques de Fuerza Bruta (Brute-Force Protection)

Riesgo actual: Cualquier atacante con un script automatizado podría enviar miles de peticiones por segundo al endpoint `/api/auth/login` probando combinaciones de contraseñas.

> **Medida requerida:** Implementar *Rate Limiting* en el backend mediante **Bucket4j** o un filtro de Spring Security. Si una dirección IP o cuenta falla 5 veces consecutivas en menos de 2 minutos, se le impone un bloqueo temporal de 15 minutos.

B. Criptografía y Almacenamiento Seguro de Contraseñas

Auditoría: Verificar que todas las contraseñas persistidas en la tabla `usuarios` se encuentren cifradas con el algoritmo **BCrypt (strength 10 u 11)** con sal (*salt*) criptográfica única generada por usuario. Nunca se debe almacenar ni transmitir contraseñas en texto plano ni con algoritmos obsoletos (MD5, SHA-1).

C. Seguridad de Tokens de Sesión (JWT)

Buenas prácticas de arquitectura: Actualmente el token JWT reside en `localStorage` del navegador. Para alcanzar el estándar bancario o de salud:

- Establecer una vida útil corta para el Access Token (ej. 15 a 30 minutos).
- Implementar Refresh Token almacenado en una **Cookie HttpOnly, Secure y SameSite=Strict**, impidiendo que cualquier script malicioso (Cross-Site Scripting / XSS) pueda acceder al token de autenticación.

D. Sanitización e Inyección SQL

Diagnóstico positivo: Al utilizar *Spring Data JPA / Hibernate* con mapeo relacional de entidades y consultas tipadas (JPQL / Criteria), el sistema está protegido por defecto contra inyecciones SQL tradicionales. No obstante, se debe mantener la regla estricta de no concatenar cadenas directas en consultas nativas.

4. Base de Datos e Infraestructura Productiva

> **Peligro crítico: Desactivar Hibernate ddl-auto=update en producción.**
> En el archivo `application.properties` actualmente figura `spring.jpa.hibernate.ddl-auto=update`. En un entorno de producción con datos reales de clientes, esta configuración es peligrosa porque una modificación accidental en una entidad Java puede alterar tablas en vivo, renombrar columnas o causar pérdida de datos.

Plan de migración segura de Base de Datos:

1. Fijar `spring.jpa.hibernate.ddl-auto=validate` en el perfil de producción.
2. Incorporar **Flyway** o **Liquibase** para que todas las modificaciones de esquema (tablas, índices, columnas) se ejecuten mediante scripts SQL versionados y auditables (ej. `V1__crear_tablas_iniciales.sql`).

Estrategia de Backups Automatizados (Regla 3-2-1):

- **Frecuencia:** Volcado diario automático (`mysqldump`) programado a las 03:00 AM mediante tarea Cron.
- **Cifrado y Almacenamiento:** Archivos comprimidos (`.sql.gz`) cifrados con GPG y sincronizados a un almacenamiento remoto seguro (AWS S3, Google Cloud Storage o servidor de contingencia).
- **Retención:** Mantener backups diarios por 30 días y mensuales por 1 año.

5. Estrategia de Despliegue y DevOps

Para evitar el clásico problema de "*en mi máquina funciona*" y asegurar alta disponibilidad, el despliegue se debe estructurar en contenedores independientes orquestados por Docker Compose:

Contenedor	Base	Puerto Interno	Función
atlas-frontend	Node.js 20 Alpine (Next.js Standalone)	3000	Renderizado SSR y portal PWA para socios y admin.
atlas-backend	Eclipse Temurin JDK 17/21	8080	API REST segura Spring Boot con autenticación JWT.
atlas-database	MySQL 8.0 Server Oficial	3306	Persistencia de datos con volumen montado en disco host.
atlas-proxy	Nginx / Caddy Reverse Proxy	80 / 443	Terminación SSL/TLS (HTTPS), compresión Gzip y cortafuegos.

6. Hoja de Ruta Priorizada hacia Producción (Cronograma de 3 Semanas)

Semana	Objetivos Principales	Entregables Técnicos	Impacto
Semana 1 Hardening & Seguridad	• Implementación de Rate Limiting. • Validación de BCrypt en 100% de usuarios. • Aislamiento de variables de entorno (.env). • CORS estricto restringido al dominio web.	• Filtro anti-fuerza bruta activo. • Archivo `application-prod.properties`. • Claves JWT generadas fuera del repositorio.	**Seguridad Crítica**
Semana 2 Base de Datos & DevOps	• Integración de Flyway para migraciones. • Creación de Dockerfiles multi-stage. • Orquestación con Docker Compose. • Script de backup diario automático.	• `docker-compose.yml` funcional. • Migraciones SQL versionadas. • Script `backup_atlas_gym.sh` en cron.	**Estabilidad y Resiliencia**
Semana 3 Staging & Go-Live	• Despliegue en servidor VPS / Cloud. • Emisión de certificado SSL Let's Encrypt. • Pruebas de estrés y carga (50 usuarios sim.). • Pruebas de aceptación de usuario (UAT).	• Dominio con HTTPS activo. • Monitoreo de logs con Spring Actuator. • **Pase a Producción Oficial**.	**Salida a Producción**

7. Dictamen Final de la Consultoría

La plataforma **ATLAS Gym** cuenta con una arquitectura de software moderna, código estructurado con TypeScript estricto, una experiencia visual refinada y una suite sólida de tests que garantizan que el negocio opera como se espera.

El tiempo restante de **2 a 3 semanas** no representa desarrollo de funcionalidades desde cero, sino la colocación del **blindaje de ingeniería** necesario para garantizar que el sistema nunca pierda un pago, resista intentos de acceso no autorizado y opere con disponibilidad ininterrumpida las 24 horas del día.

Documento elaborado por el Equipo de Arquitectura y Auditoría de Software · ATLAS Gym Platform · Octubre 2026